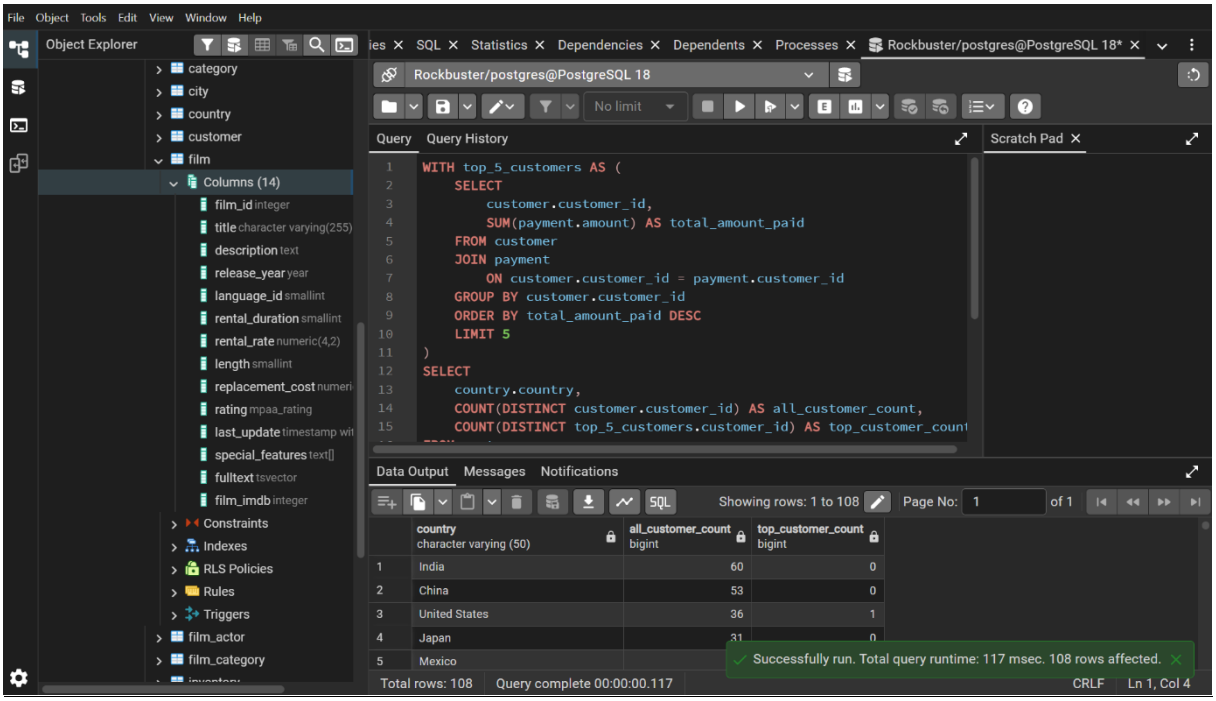
Common Expression Tables

Answer the business questions from steps 1 & 2 of the previous task using CTEs

1. Rewrite your queries from steps 1 and 2 the previous task as CTEs.
2. Write 2 to 3 sentences explaining how you approached this step, for example, what you did first, second, and so on.



Query Applied:

```
WITH top_5_customers AS (
  SELECT
    customer.customer_id,
    SUM(payment.amount) AS total_amount_paid
  FROM customer
  JOIN payment
    ON customer.customer_id = payment.customer_id
  GROUP BY customer.customer_id
  ORDER BY total_amount_paid DESC
```

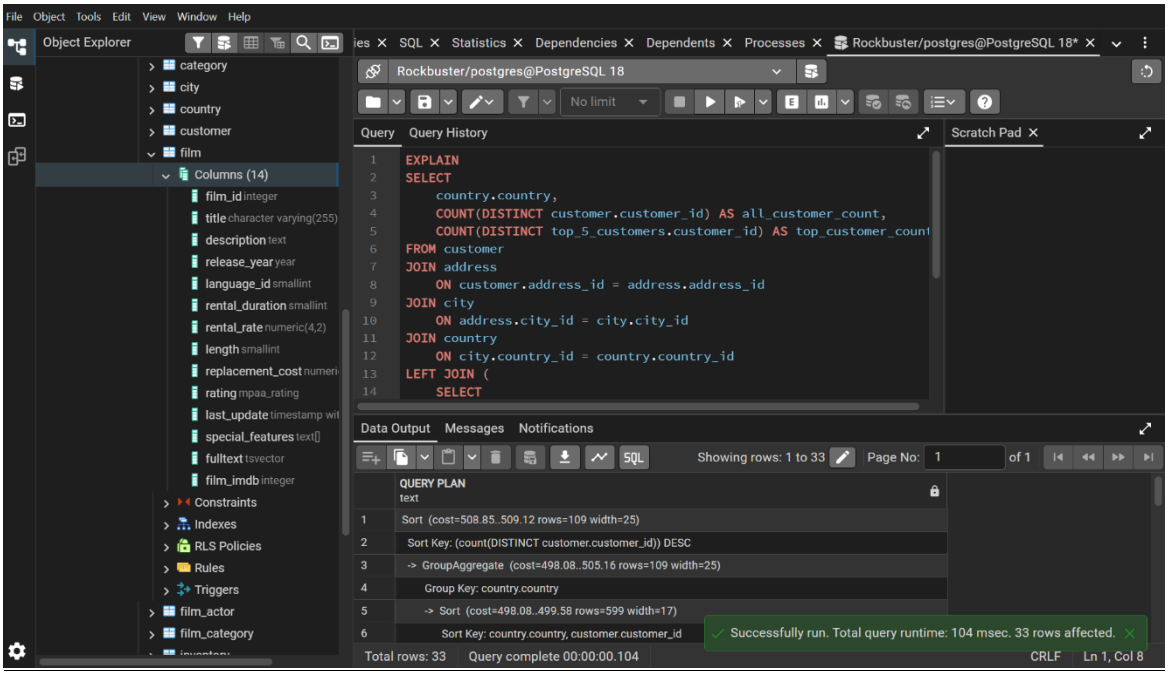
```
  LIMIT 5
)
SELECT
  country.country,
  COUNT(DISTINCT customer.customer_id) AS all_customer_count,
  COUNT(DISTINCT top_5_customers.customer_id) AS top_customer_count
FROM customer
JOIN address
  ON customer.address_id = address.address_id
JOIN city
  ON address.city_id = city.city_id
JOIN country
  ON city.country_id = country.country_id
LEFT JOIN top_5_customers
  ON customer.customer_id = top_5_customers.customer_id
GROUP BY country.country
ORDER BY all_customer_count DESC;
```

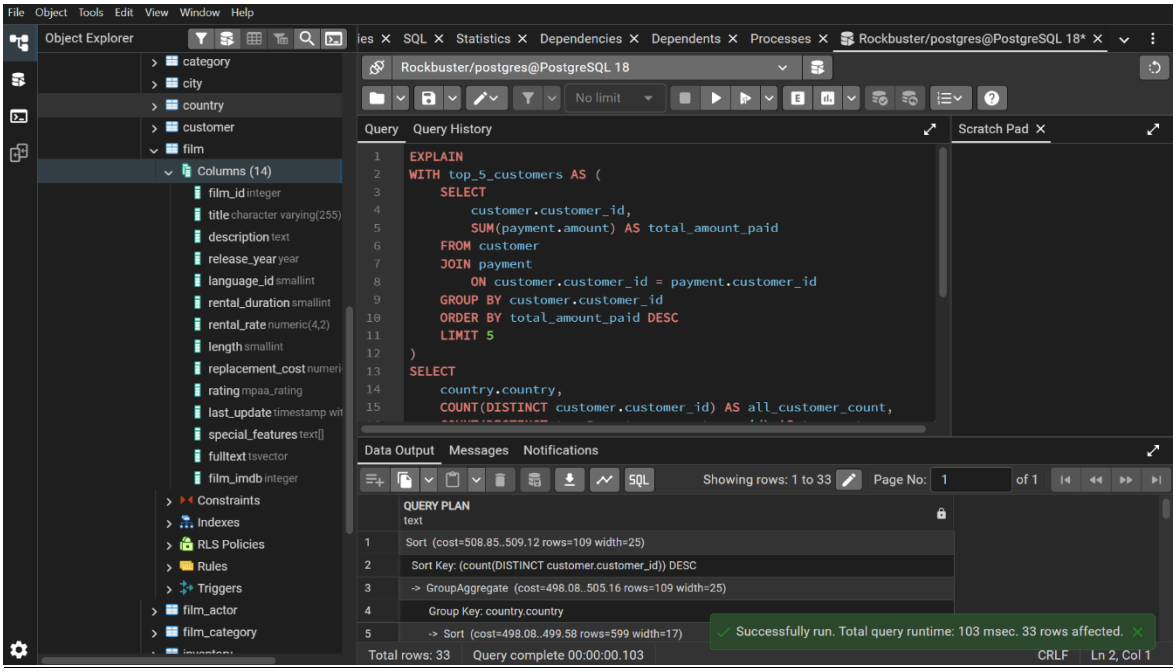
Answers:

To rewrite the query using a CTE, I first moved the inner subquery into a WITH clause and assigned it the name top_5_customers. This allowed me to isolate the logic that calculates the highest paying customers before running the main query. I then reused this temporary result set in the LEFT JOIN exactly as I had done with the subquery. Structuring the query this way improved readability and made the overall logic easier to follow. The results are the same as those from Exercise 3.8.

Compare the performance of your CTEs and subqueries.

1. Which approach do you think will perform better and why?
2. Compare the costs of all the queries by creating query plans for each one.





Answers:

To compare performance, I generated query plans using the EXPLAIN command for both versions of the query — the subquery version and the CTE version. The estimated costs shown in the query plans were effectively the same for both queries. Both plans displayed similar operations, including Sort and GroupAggregate, indicating that PostgreSQL's optimizer treated the two query structures similarly.

The EXPLAIN command gives you an *estimated* cost. To find out the actual speed of your queries, run them in pgAdmin 4. After you've run each query, a popup window will display its speed in milliseconds.

After running both queries, the execution times were also nearly identical. The subquery version executed in 104 ms, while the CTE version executed in 103 ms. This difference is negligible and suggests that, for this particular query, using a CTE did not produce any meaningful performance improvement.

Based on both the estimated costs and actual runtimes, neither approach clearly outperformed the other. This is because modern PostgreSQL versions often inline CTEs when possible, meaning the database engine optimizes them similarly to subqueries. As a result, performance differences are often minimal unless working with very large datasets or complex logic.

Did the results surprise you? Write a few sentences to explain your answer.

Yes, the results did surprise me. Before running the queries, I expected the CTE version to have a noticeably lower cost because the logic felt more structured and easier to read. However, after comparing the execution plans with EXPLAIN, I learned that readability improvements do not necessarily translate into better performance. I now know how PostgreSQL optimizes queries and why it is important to validate assumptions using query plans rather than relying on intuition.

While CTEs are often assumed to be slower or faster depending on the database system, PostgreSQL's query planner is sophisticated enough to optimize both CTEs and subqueries efficiently. The nearly identical runtimes confirm that query structure does not always have a significant impact on performance.

Write 1 to 2 paragraphs on the challenges you faced when replacing your subqueries with CTEs.

Answer:

One challenge I faced when converting the subqueries into CTEs was ensuring that the logic of the original query remained unchanged. Although CTEs and subqueries can produce the same results, restructuring the query required careful attention to aliases, join conditions, and column references. Small mistakes, such as referencing a column from the wrong scope or forgetting to apply grouping at the correct level, easily caused errors or incorrect outputs.

Another difficulty was understanding how the flow of the query changes conceptually. With subqueries, the logic is embedded directly inside the main statement, whereas CTEs separate different parts of the computation into named blocks. While this improves readability, it requires clearer planning of the query steps. I had to think more deliberately about which calculations belonged in the CTE versus the outer query, especially when dealing with aggregations and joins. Despite these challenges, the final CTE version was easier to read and interpret once completed.